UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Varrel Fernando / 2428240068
Kelas	SI5B
Judul project / topik API	RESTful API Museum Collection
Tautan repositori	https://github.com/Lyvarion-18/tugas1-restful-2428240068
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	22-10-2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

> **Sebelum mengambil screenshot Runner**
> Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:
RESTful API Museum Collection merupakan API yang digunakan untuk mengelola data koleksi museum, meliputi nama koleksi, kategori, periode sejarah, asal daerah, dan tahun ditemukan. API ini dikembangkan menggunakan Node.js dan Express.js dengan menyediakan operasi CRUD (Create, Read, Update, Delete) untuk mengakses dan mengelola data koleksi museum.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/museum-collections	Tidak	200	Mengambil seluruh data koleksi museum
2	GET	/museum-collections/:id	Tidak	200	Mengambil data koleksi berdasarkan ID
3	GET	/museum collections?periode=Sriwijaya	Tidak	200	Memfilter koleksi berdasarkan periode sejarah
4	POST	/museum-collections	Tidak	201	Menambahkan data koleksi museum
5	PUT	/museum-collections/:id	Tidak	200	Memperbarui data koleksi berdasarkan ID
6	DELETE	/museum-collections/:id	Tidak	200	Menghapus data koleksi berdasarkan ID
7					
8					

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

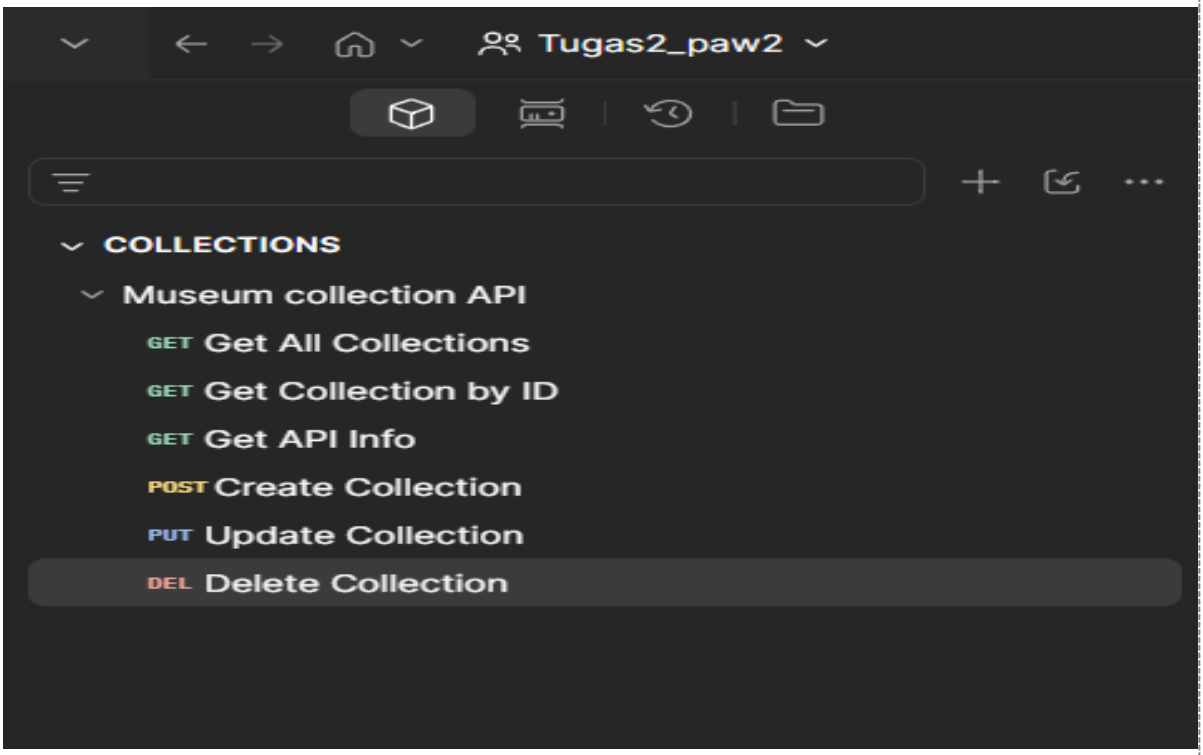
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Menyimpan alamat dasar server API
2	Collection_id	Diisi otomatis oleh script	Menyimpan ID koleksi museum yang dibuat melalui POST
3			
4			

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:

Collection Museum collection API berisi enam request yang digunakan untuk menguji fitur pada RESTful API Museum Collection. Setiap request memiliki fungsi yang berbeda, mulai dari melihat informasi API dan data koleksi, mencari koleksi berdasarkan ID, menambahkan data baru, mengubah data, hingga menghapus data koleksi.

Semua request dikumpulkan dalam satu Collection agar lebih mudah dikelola dan dijalankan saat proses pengujian menggunakan Postman.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	Get API Info	GET	/	Menampilkan informasi API dan daftar endpoint yang tersedia.
2	Get All Collections	GET	/museum-collections	Menampilkan seluruh data koleksi museum.
3	Get Collection by ID	GET	/museum-collections/:id	Mengambil data koleksi berdasarkan ID.
4	Create Collection	POST	/museum-collections	Menambahkan data koleksi baru.
5	Update Collection	PUT	/museum-collections/:id	Mengubah data koleksi berdasarkan ID.
6	Delete Collection	DELETE	/museum-collections/:id	Menghapus data koleksi berdasarkan ID.

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	Get All Collections	200ok	Memeriksa respons berupa array dan field koleksi yang diperlukan.
2	Get Collection by ID	200ok	Memeriksa respons berupa objek JSON dan ID koleksi bernilai 1
3	Get API Info	200ok	Memeriksa nama, NIM, topik, dan daftar endpoint API.
4	Create Collection	201created	Memeriksa status success dan keberadaan ID koleksi yang dibuat.
5	Update Collection	200ok	Memeriksa status success dan nama koleksi yang sudah diperbarui.
6	Delete Collection	200ok	Memeriksa status success, pesan konfirmasi penghapusan, dan data: null.
7			
8			
9			
10			

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
Test script request GET: Get API Info
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response has correct identity", function () {
    const data = pm.response.json();

    pm.expect(data).to.have.property("nama", "Varrel Fernando");
    pm.expect(data).to.have.property("nim", "2428240068");
});

pm.test("Response has endpoint list", function () {
    const data = pm.response.json();

    pm.expect(data).to.have.property("topik", 13);
    pm.expect(data.endpoints).to.be.an("array");
    pm.expect(data.endpoints.length).to.be.greaterThan(0);
});
```

Penjelasan:

Script tersebut memiliki tiga kelompok pengujian. Pertama, memastikan API mengembalikan status HTTP 200. Kedua, memeriksa apakah nama dan NIM pada respons sesuai dengan identitas yang diharapkan. Ketiga, memeriksa nilai topik dan memastikan properti endpoints berupa array yang tidak kosong.

Dengan demikian, pengujian tidak hanya memastikan API berhasil merespons, tetapi juga memeriksa kesesuaian isi responsnya.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
Test script request POST: Create Collection
pm.test("Status code is 201", function () {
    pm.response.to.have.status(201);
});

pm.test("Response status is success", function () {
    const data = pm.response.json();
    pm.expect(data.status).to.eql("success");
});

pm.test("Created collection has an ID", function () {
    const data = pm.response.json();
    pm.expect(data.data.id).to.exist;

    pm.environment.set("collection_id", data.data.id);
});
```

Penjelasan:

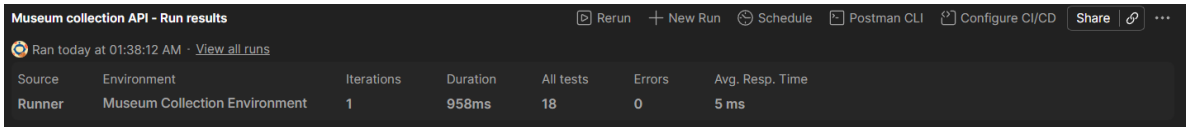
Script POST *Create Collection* digunakan untuk memastikan data koleksi berhasil ditambahkan dengan status 201 Created dan respons berstatus success. ID koleksi yang berhasil dibuat disimpan ke variabel collection_id untuk digunakan pada request *Update Collection* dan *Delete Collection*. Ketiga assertion berhasil dijalankan dengan hasil **3/3 test passed**.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:
Collection Runner digunakan untuk menjalankan seluruh request dalam Collection secara otomatis. Hasil pengujian menunjukkan bahwa 18 test berhasil (*passed*) dan tidak ada test yang gagal (*failed*). Hal ini menunjukkan bahwa seluruh assertion yang dijalankan berhasil dipenuhi.

4.2 Rekap angka hasil

Nama Collection	Museum collection API
Environment yang dipakai	Museum Collection Environment
Jumlah request dijalankan	6
Jumlah test lulus	18
Jumlah test gagal	0
Tanggal dan jam pengujian	22 Oktober 2026, 01.38 WIB

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Get Collection by ID	TypeError: "" is not a function	Script pengujian mengalami error saat dijalankan	Memperbaiki script hingga 3/3 test berhasil.
2	Update Collection	404 Not Found	Data dengan ID yang diminta tidak ditemukan.	Memastikan ID koleksi dan variabel collection_id digunakan dengan benar.

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.



Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/Lyvarion-18/tugas1-restful-2428240068
Lokasi berkas Collection (.json)	Museum collection API.postman_collection.json
Lokasi berkas Environment (.json)	C:\Users\User\OneDrive\Documents
Berkas Environment tidak memuat kunci rahasia sungguhan	[✔] Ya [ ] Tidak
Repositori dapat dibuka dosen	[✔] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:
Assertion yang paling berguna menurut saya adalah pemeriksaan status code dan isi respons JSON, karena keduanya membantu memastikan API tidak hanya merespons, tetapi juga memberikan data yang sesuai. Contohnya, saat pengujian Update Collection menghasilkan 404 Not Found, saya mengetahui bahwa ID koleksi yang digunakan perlu diperiksa kembali.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:
Pengujian manual lebih mudah dilakukan untuk memeriksa respons satu per satu, sedangkan Collection Runner membantu menjalankan seluruh request secara otomatis dan menampilkan hasilnya sekaligus. Bagian tersulit bagi saya adalah menulis test script dan memastikan variabel collection_id tersimpan serta digunakan dengan benar oleh request berikutnya.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	✔
2	Setiap endpoint API saya ada di Collection	✔
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	✔
4	Setiap request punya assertion status code dan struktur JSON	✔
5	Nama setiap test berupa kalimat yang menyatakan harapan	✔
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	✔
7	Server dinyalakan ulang sebelum Runner dijalankan	✔
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	✔
9	Berkas Environment tidak memuat kunci rahasia sungguhan	✔
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	✔

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen